Hoodly

Descriptif fonctionnel

1. Introduction

Hoodly est une plateforme collaborative de quartier développée dans le cadre de notre projet annuel de 3ème année à l'ESGI. L'idée est simple : créer un espace numérique où les habitants d'un même quartier peuvent s'entraider, organiser la vie locale et rester connectés entre eux.

Le principe repose sur la notion de zones géographiques. Chaque utilisateur rejoint un quartier, et c'est au sein de cette zone qu'il peut accéder aux fonctionnalités de la plateforme : échanger des services avec ses voisins, signer des documents en ligne, participer à des événements, échanger via une messagerie, voter sur des sujets de quartier ou encore signaler des incidents.

Pour fonctionner, l'application s'appuie sur trois interfaces : une application web destinée aux habitants, un back-office d'administration pour gérer la plateforme, et une application desktop Java réservée à l'administrateur pour la gestion des incidents en mode déconnecté.

2. Acteurs du système

On distingue trois types d'utilisateurs sur la plateforme, chacun avec des permissions spécifiques.

L'**habitant** (USER) est l'utilisateur standard. Il peut consulter les zones, demander à en rejoindre une, publier ou répondre à des annonces de services, participer aux événements, envoyer des messages, voter aux sondages, signaler des incidents et signer des documents. C'est l'acteur principal de la plateforme.

Le **modérateur** (MODERATEUR) hérite de toutes les permissions de l'habitant, avec en plus la capacité de valider ou refuser les demandes d'adhésion aux quartiers et de modérer les contenus. On le voit comme un habitant avec des responsabilités supplémentaires, pas comme un super-admin.

L'**administrateur** (ADMIN) a accès à tout. Il gère les zones, les utilisateurs, les incidents, peut créer des sondages et accède au dashboard Java. C'est lui qui a la main sur la structure de la plateforme dans son ensemble.

La hiérarchie est donc : ADMIN > MODERATEUR > USER. Un admin peut tout faire, un modérateur peut faire tout ce qu'un habitant fait plus la modération, et un habitant a accès aux fonctionnalités de base.

3. Application web - Fonctionnalités détaillés

3.1 Authentification et sécurité

Pour l'authentification, on a choisi d'utiliser Auth0 plutôt que de tout développer nous-mêmes. C'est un service d'identité reconnu qui gère la connexion, le SSO et le MFA, et ça nous évite de réinventer la roue sur un sujet aussi sensible que la sécurité.

Le fonctionnement est le suivant : quand un utilisateur se connecte, il est redirigé vers Auth0, saisit ses identifiants (email/mot de passe ou Google OAuth), et Auth0 lui renvoie un token JWT. C'est ce token qu'on envoie au backend à chaque requête, et c'est le backend qui le vérifie via les clés publiques JWKS d'Auth0.

À chaque première connexion d'un utilisateur, on synchronise son profil Auth0 avec notre base MongoDB. Ça nous permet d'avoir notre propre copie des données utilisateur (nom, email, photo) sans dépendre uniquement d'Auth0. Concrètement, le backend vérifie si l'utilisateur existe dans MongoDB via son Auth0 ID, le crée si c'est sa première connexion, ou met à jour ses infos si elles ont changé.

Pour les actions sensibles, on prévoit le MFA (authentification multi-facteurs). Ça concernera : la connexion elle-même, la modification d'email ou de téléphone, le changement de mot de passe et la signature de documents. Auth0 gère ça nativement avec des codes TOTP, donc pas besoin de coder nous-mêmes.

L'application Java Desktop utilisera le même compte Auth0 mais avec un flux PKCE (Proof Key for Code Exchange). C'est la méthode recommandée pour les applications desktop : pas besoin de stocker un secret client côté application, ce qui est plus sécurisé. L'admin se connecte une fois et a accès aux deux plateformes avec le même compte.

Pour la gestion des rôles, on les stocke directement dans les claims du JWT. Comme ça le backend n'a pas besoin de faire une requête en base à chaque requête pour vérifier les permissions, il lit directement le rôle depuis le token. Un Guard NestJS intercepte chaque requête, vérifie le token, extrait le rôle et le compare aux permissions requises par l'endpoint.

3.1 Gestion des utilisateurs

Côté gestion des utilisateurs, on fait quelque chose de relativement classique mais qu'il fallait penser proprement. L'admin a accès à une vue liste de tous les utilisateurs inscrits, avec un système de pagination pour ne pas charger tout d'un coup, et des filtres pour chercher par nom, rôle ou statut (actif/inactif).

Pour chaque utilisateur, l'admin peut consulter le profil complet, modifier son rôle (le faire passer de user à modérateur par exemple) et changer son statut actif. Si un utilisateur pose problème, on peut le désactiver plutôt que de le supprimer directement, comme ça on garde une trace de ce qu'il a fait sur la plateforme.

La suppression existe aussi, mais elle implique l'anonymisation des données pour rester conforme au RGPD. Concrètement, quand on supprime un compte, on garde les données utiles à la plateforme (les messages envoyés, les événements créés, etc.) mais on remplace les infos personnelles par des données anonymisées. Comme ça l'historique ne se retrouve pas avec des trous dans les conversations ou les événements.

Une idée qu'on a eue : ajouter un système de signalement d'utilisateur. Si un habitant se comporte mal (arnaques sur les services, messages inappropriés), les autres utilisateurs peuvent le signaler, et un modérateur ou admin

peut intervenir. Cela s’inscrit dans la logique de modération de la plateforme.

3.3 Modélisation géographique

Les zones sont le cœur de la plateforme. Chaque fonctionnalité de Hoodly est rattachée à un quartier, donc il fallait que la gestion des zones soit solide dès le départ.

Le principe est simple : un quartier, c'est un polygone dessiné sur une carte. Côté technique, on utilise le format GeoJSON natif de MongoDB pour stocker les coordonnées du polygone, et Mapbox côté front pour l'affichage et le dessin. L'admin peut créer une zone directement en dessinant le périmètre sur la carte, en donnant un nom et une ville. Le polygone est ensuite stocké en base et servira de référence pour toutes les fonctionnalités liées au quartier.

Mais on ne peut pas laisser l'admin tout faire seul. Les habitants ont aussi la possibilité de proposer la création d'un nouveau quartier. Ils remplissent un formulaire avec le nom souhaité, la ville, le code postal et une description. La demande part en statut "en attente", l'admin reçoit une notification et peut accepter ou refuser, avec un commentaire pour expliquer son choix. Si c'est accepté, la zone est créée automatiquement et l'habitant qui l'a proposée en est le premier membre. Pour éviter le spam, on limite à une demande en attente par utilisateur.

Pour rejoindre un quartier existant, c'est un peu plus encadré. L'habitant sélectionne la zone qu'il veut rejoindre et doit fournir un justificatif de domicile ainsi qu'une pièce d'identité. C'est un modérateur ou un admin qui vérifie les pièces et valide ou refuse l'adhésion. On pourrait envisager un système de vérification automatique plus tard (OCR sur les documents, par exemple), mais pour l'instant on part sur une validation manuelle.

Le cas des limites est intéressant. Que se passe-t-il si un utilisateur habite à la frontière entre deux quartiers ? Notre approche : quand on localise un utilisateur et que son adresse tombe dans le rayon de deux zones adjacentes, on lui propose les deux et il choisit laquelle il veut rejoindre. Si son adresse ne tombe dans aucune zone, on lui propose d'en créer une nouvelle demande. L'algorithme qu'on utilisera est le ray casting pour tester si un point GPS est à l'intérieur d'un polygone.

La carte interactive via Mapbox servira à plusieurs choses : visualiser les quartiers, afficher les incidents signalés, voir les événements à proximité. C'est un vrai plus pour l'expérience utilisateur par rapport à une simple liste.

3.4 Services entre voisins

C'est la fonctionnalité la plus emblématique de Hoodly. L'idée est de permettre aux habitants de s'entraider au quotidien en échangeant des services entre voisins.

Le système repose sur des annonces. Un habitant publie une annonce qui peut être soit une offre ("je propose du babysitting"), soit une demande ("j'ai besoin d'aide pour déménager"). Chaque annonce précise si le service est gratuit ou payant.

Pour les services gratuits, c'est simple : les voisins se débrouillent entre eux, pas besoin de complexité supplémentaire. C'est le principe de base du bon voisinage, mais rendu visible et organisé.

Pour les services payants, on a mis en place un système de points. Plutôt que de gérer de l'argent réel (ce qui impliquerait de la réglementation et des paiements sécurisés), on utilise une monnaie virtuelle interne. Chaque service payant donne un certain nombre de points au prestataire et débite le demandeur. Le calcul se base sur la durée du service multipliée par un coefficient qui dépend du type de service. Par exemple, du babysitting à 1.33

points par heure (3 heures = 4 points), un cours particulier à 2 points par heure, du jardinage à 1 point par heure. Les coefficients sont paramétrables par l'admin pour ajuster au fil du temps.

Quand un service payant est accepté, un contrat est automatiquement généré. Le prestataire et le demandeur doivent signer le contrat avant que le service ne soit considéré comme actif. La signature se fait via notre module de documents (voir dans la section suivante).

Une idée qu'on aimerait ajouter : un système de notation et de confiance. Après un service effectué, les deux utilisateurs pourraient se noter mutuellement. Les voisins les mieux notés apparaîtraient en premier dans les résultats de recherche, et le moteur Neo4j utiliserait ces notes pour recommander des voisins fiables. Ça rejoint la fonctionnalité de recommandations que nous verrons plus tard.

3.5 Documents et signatures en ligne

Quand un service payant est accepté entre deux voisins, Hoodly génère automatiquement le contrat de service. Le prestataire et le demandeur n'ont rien à rédiger, c'est la plateforme qui s'en charge à partir des informations du service (type, durée, points, identité des deux parties).

Le contrat est d'abord envoyé au prestataire. Il consulte le document, confirme son identité avec le MFA et appose sa signature numérique. Une fois signé par le premier, le contrat est transmis au demandeur qui fait la même démarche : consultation, MFA, signature. Après les deux signatures, le contrat est automatiquement archivé.

À chaque étape, un horodatage est enregistré. On sait exactement qui a signé quoi et quand, ce qui garantit la traçabilité. Les contrats archivés ne peuvent plus être modifiés, ils servent de preuve en cas de désaccord entre voisins.

Le système de signatures est pensé pour être extensible. Pour l'instant il sert aux contrats de services, mais on pourrait l'utiliser pour d'autres types de documents : des autorisations de travaux, des accords de prêt de matériel, des formulaires administratifs du quartier. L'admin pourrait créer des templates de documents depuis le back-office, et la plateforme les enverrait aux habitants concernés avec le même processus de signature séquentielle.

3.6 Évènements et activités

La vie de quartier, c'est aussi des moments partagés. Le module événements permet aux habitants de créer et gérer des activités communautaires : soirées, ateliers, collectes, réunions de quartier, barbecues, quoi que ce soit.

N'importe quel habitant membre d'une zone peut créer un événement. Il remplit les infos classiques : titre, description, date de début et de fin, lieu. L'événement est publié dans sa zone et les voisins sont notifiés. La création est simple et rapide, on veut pas que les gens passent 20 minutes à remplir un formulaire pour proposer un apéro.

Pour la participation, on a prévu deux niveaux. Le premier, c'est l'inscription classique : on clique sur "je participe" et c'est actif. Le deuxième, c'est ce qu'on appelle le swipe d'intérêt. L'idée, c'est qu'un voisin puisse indiquer qu'il est intéressé sans s'engager formellement. Il swipe, ça enregistre son intérêt, mais il n'est pas compté comme inscrit. C'est utile pour les gens qui hésitent ou qui veulent juste voir si l'événement intéresse du monde avant de s'y mettre.

La vraie plus-value, c'est le moteur de recommandations. Grâce à Neo4j, on peut analyser les interactions entre voisins : qui a participé à quels événements ensemble, qui a échangé des services avec qui, qui a montré de l'intérêt pour quoi. Sur cette base, la plateforme peut suggérer des événements pertinents à chaque utilisateur. Par exemple : "3 de vos voisins participent à cet atelier cuisine, peut-être que ça vous intéresse aussi." C'est un vrai atout par rapport à une simple liste d'événements à scroller.

Quelque chose qu'on aimerait ajouter : un système d'événements récurrents. Si un voisin organise un jeu de cartes tous les jeudis, il devrait pouvoir créer l'événement une seule fois avec une répétition hebdomadaire plutôt que de recréer chaque semaine. Aussi, une vue calendrier en plus de la liste classique, pour mieux visualiser les activités de la semaine ou du mois.

3.7 Messagerie multimédia sécurisée

La messagerie, c'est simple : on veut que les voisins puissent s'écrire entre eux. Le système fonctionne en messages privés, avec support du texte classique.

Côté multimédia, on prévoit l'envoi de photos et éventuellement des messages vocaux si on a le temps. Pour l'accès au micro et à la caméra, on utilisera l'API native getUserMedia() du navigateur qui permet de capturer l'audio et la vidéo directement depuis le front.

La communication temps réel passe par Socket.io (WebSocket). Les messages arrivent instantanément si le destinataire est en ligne, et sont stockés en base s'il est déconnecté. Le statut online/offline des utilisateurs est visible en temps réel aussi.

3.8 Votes et sondages

Le module de votes permet de prendre des décisions collectives au sein d'un quartier. L'idée est de donner aux habitants un moyen structuré de s'exprimer sur des sujets de vie locale : installer un nouveau playground, organiser une fête de quartier, changer les horaires de la buanderie collective, etc...

La création de sondages est réservée aux admins et modérateurs. Un sondage a un titre, une description, plusieurs options de réponse, et des dates de début et de fin. Une fois publié, les habitants de la zone concernée reçoivent une notification et peuvent voter.

Pour empêcher les abus, on gère le problème du double vote : un utilisateur ne peut voter qu'une seule fois par sondage. Côté technique, on stocke chaque vote dans une collection dédiée avec le couple (sondageId, utilisateurId), et on vérifie l'existence avant d'accepter un nouveau vote.

Les résultats sont affichés en temps réel. Dès qu'un habitant vote, les pourcentages se mettent à jour pour tout le monde. C'est visuellement plus engageant et ça incite les gens à participer quand ils voient que le sondage bouge.

Quand la date de fin est atteinte, le sondage se ferme automatiquement et les résultats finaux sont archivés. L'admin pourrait consulter l'historique des sondages passés à tout moment.

3.9 Gestion des incidents

Les incidents font partie de la vie de quartier. Un dégât dans les parties communes, du bruit excessif, un problème de sécurité, un lampadaire cassé : autant de choses que les habitants doivent pouvoir signaler rapidement.

Le mécanisme est simple : un habitant clique sur "Signaler un incident", remplit un formulaire avec le type de problème (bruit, dégradation, sécurité, autre), une description, éventuellement une photo et un niveau de priorité. L'incident est créé dans la zone concernée et l'admin est notifié.

La gestion des statuts suit un cycle logique : signalé → en cours → résolu → fermé. Un incident signalé peut passer en cours quand quelqu'un s'en occupe, puis en résolu quand c'est traité, puis fermé quand c'est confirmé. On prévoit aussi la réouverture : un admin peut remettre un incident "résolu" en "en cours" si le problème revient.

La vue admin permet de filtrer les incidents par statut, priorité ou zone. C'est utile quand il y a beaucoup de signalements, l'admin peut se concentrer d'abord sur les urgents.

Côté technique, cette fonctionnalité est aussi liée à l'application Java Desktop. L'admin peut consulter et gérer les incidents depuis le client lourd.

3.10 Recommandations (Neo4j)

C'est la fonctionnalité qui fait de Hoodly plus qu'une simple plateforme de quartier. Le moteur de recommandations s'appuie sur Neo4j, une base de données graphe, pour analyser les interactions entre voisins et proposer des suggestions pertinentes.

Le principe est de modéliser les relations entre utilisateurs sous forme de graphe. Quand deux voisins échangent un service, on crée une relation HELPED dans le graphe. Quand ils participent au même événement, on crée une relation ATTENDED. Quand un utilisateur montre de l'intérêt pour un type de service, on crée une relation INTERESTED_IN. Au fil du temps, le graphe se construit et reflète la vie sociale du quartier.

Sur cette base, le moteur peut faire trois types de recommandations. D'abord, des voisins fiables : si un voisin a déjà aidé beaucoup de monde et qu'il propose un service dont j'ai besoin, il sera recommandé en priorité. Ensuite, des événements pertinents : si mes voisins les plus proches (ceux avec qui j'ai le plus d'interactions) participent à un événement, il y a des chances que ça m'intéresse aussi. Enfin, des services compatibles : si j'ai besoin d'aide en jardinage et que mon voisin a aidé quelqu'un d'autre dans le même domaine, on peut me le suggérer.

Côté technique, les requêtes se font en Cypher, le langage de requête natif de Neo4j. Le backend fait des appels à Neo4j en parallèle de MongoDB : MongoDB gère les données métier (utilisateurs, services, événements) et Neo4j gère uniquement les relations et les recommandations. C'est deux bases qui travaillent ensemble, chacune sur son terrain.

Une idée qu'on aimerait explorer : pondérer les recommandations avec le temps. Une interaction vieille de six mois a moins de poids qu'une interaction de la semaine dernière. Comme ça les suggestions restent fraîches et reflètent la situation actuelle du quartier.

3.11 Conformité RGPD

Étant une plateforme qui manipule des données personnelles (noms, emails, adresses, documents signés, conversations), le respect du RGPD n'est pas négociable. On l'a intégré dans la conception plutôt que de le traiter comme une contrainte ajoutée après coup.

La plateforme doit répondre à quatre droits fondamentaux. Le droit d'accès : un utilisateur peut demander l'ensemble de ses données personnelles et les exporter dans un format standard (JSON). Le droit de rectification : il peut modifier ses informations directement depuis son profil. Le droit de suppression : quand un utilisateur supprime son compte, ses données personnelles sont anonymisées. On ne supprime pas tout car il faut garder la cohérence de la plateforme (un événement créé par un utilisateur supprimé doit rester visible), mais on remplace les infos perso par des données anonymes. Le droit à la portabilité : les données sont exportables dans un format exploitable en dehors de Hoodly.

Au niveau technique, on gère ça côté backend avec des endpoints dédiés. L'export des données est un process qui compile tout ce qui est lié à un utilisateur (profil, messages, services, votes, documents signés) dans un fichier téléchargeable. La suppression est plus délicate parce qu'il faut nettoyer les données sans casser les relations en base : un message envoyé reste mais l'expéditeur devient "utilisateur supprimé", un service créé reste mais l'auteur est anonymisé.

On fait aussi attention au stockage des justificatifs (pièce d'identité, justificatif de domicile). Ce sont des données sensibles, on les conserve uniquement le temps de la validation de l'adhésion et on les supprime après.

3.12 Internationalisation

Le sujet impose que l'application soit multilingue. On prévoit un support en français et en anglais au minimum, avec la possibilité d'ajouter d'autres langues par la suite.

Côté technique, on utilise une librairie de gestion des traductions côté front (type i18next avec React). Tous les textes visibles par l'utilisateur sont externalisés dans des fichiers de traduction, un par langue. Les erreurs de validation, les labels des boutons, les notifications, les descriptions de services, tout passe par ce système.

La langue est sélectionnée par l'utilisateur dans ses paramètres et sauvegardée dans son profil. À chaque chargement de page, l'appli charge le fichier de traduction correspondant. Si une traduction manque, on retombe sur le français par défaut.

Côté back-office admin, c'est aussi en français et anglais. Les contenus dynamiques (les annonces, les messages entre voisins) restent dans la langue dans laquelle ils ont été écrits, on ne traduit pas les échanges entre utilisateurs, ça n'aurait pas de sens.

4. Application Java Desktop

L'application Java Desktop est un client lourd réservé à l'administrateur. Son rôle principal est la gestion des incidents et des statistiques du quartier, avec la particularité de fonctionner même sans connexion internet.

Le développement se fait en Java avec JavaFX pour l'interface graphique. L'admin se connecte via le même compte Auth0 que sur le web, avec un flux PKCE sécurisé. Une fois authentifié, il a accès à un dashboard qui affiche les incidents avec des statistiques (nombre de signalés, en cours, résolus, fermés) et des graphiques pour visualiser la situation.

Pour la gestion des incidents, l'admin peut modifier le statut et la priorité des signalements directement depuis l'appli. C'est le même cycle de statuts que côté web : signalé → en cours → résolu → fermé.

Pour le mode offline-first. On utilise SQLite comme base locale embarquée. Au premier lancement, l'appli synchronise les données depuis le backend et les stocke localement. Ensuite, si la connexion est coupée, l'admin peut quand même consulter les incidents et les statistiques à partir des données locales. Il peut même ajouter ou modifier des données, elles sont mises en file d'attente avec un statut "sync_pending".

Dès que la connexion revient, la synchronisation se fait automatiquement. Les modifications locales sont envoyées au backend et comparées avec la version serveur. En cas de conflit (un même incident modifié offline et online), on applique la règle du "dernier modifié gagne" avec priorité à la version serveur, et l'admin est notifié.

L'appli gère aussi des fonctionnalités supplémentaires : des plugins (export de statistiques en PDF ou CSV, analyse sociale, calendrier local), des thèmes personnalisables (couleurs, polices, disposition), et un système de mises à jour automatiques depuis un serveur central. L'admin peut aussi désinstaller l'appli directement depuis son interface.

Le livrable final est un fichier .jar comme demandé dans le sujet.

5. Stockage des données

Le projet utilise trois bases de données, chacune avec un rôle précis. C'est un choix qu'on a justifié en fonction des besoins de chaque fonctionnalité plutôt que de tout mettre dans une seule base.

MongoDB est la base principale. C'est elle qui stocke la majeure partie des données métier : les utilisateurs, les zones (avec les polygones GeoJSON), les services, les contrats, les événements, les messages, les votes, les incidents, les documents. On a choisi MongoDB pour sa flexibilité de schéma, utile quand les structures de données varient d'une fonctionnalité à l'autre, et pour le support natif de GeoJSON indispensable pour la modélisation géographique des quartiers. Côté ODM, on utilise Mongoose avec l'intégration NestJS qui nous apporte la validation des schémas, les hooks et le typage TypeScript.

Neo4j est dédiée au moteur de recommandations. Elle ne stocke pas de données métier mais uniquement le graphe social des interactions entre voisins : qui a aidé qui, qui a participé aux mêmes événements, qui s'intéresse à quoi. Les requêtes Cypher natives de Neo4j permettent de traverser efficacement ces relations pour générer des recommandations, ce que MongoDB ferait beaucoup moins bien. Les deux bases communiquent via le backend : quand une interaction se produit dans MongoDB (un service effectué, un événement rejoint), on synchronise la relation correspondante dans Neo4j.

SQLite est la base locale de l'application Java Desktop. Elle sert uniquement pour le mode offline-first : l'admin consulte les incidents et statistiques depuis cette base quand il n'a pas de connexion. C'est une base légère, embarquée directement dans le .jar, pas besoin de serveur externe. Dès que la connexion revient, SQLite se synchronise avec le backend et MongoDB devient à nouveau la source de vérité.